

Raspberry Pi companion computer setup — swarm drone build

2026-09-21 · @Gabriele Luzzani

Scope and hardware

This is the procedure used to bring up the Raspberry Pi as a companion computer on drone 1, written so it can be repeated on drones 2 and 3. It covers everything from an unflashed SD card to a verified, boot-persistent MAVLink link between the Pi and the flight controller.

It does not cover the leader-follower layer itself. That sits on top of this infrastructure and is the same regardless of whether you use ArduPilot's built-in FOLLOW mode or write your own controller.

Hardware assumed

Component	Detail
Flight controller	Matek H7A3 (reports as MatekH7A3 , not MatekH743)
Firmware	ArduCopter 4.7.x
Companion computer	Raspberry Pi 4, 2GB
Pi OS	Raspberry Pi OS Lite 64-bit (Debian 13 Trixie)
Pi power	Dedicated 5V UBEC off the flight battery
Ground link	DroneBridge on an ESP32-C6, already working on UART4

Serial port allocation on the flight controller

Port	Used by
SERIAL1 (UART1/ TELEM1)	Raspberry Pi — the link this guide sets up
SERIAL2	GPS
SERIAL4 (UART4)	DroneBridge ESP32-C6

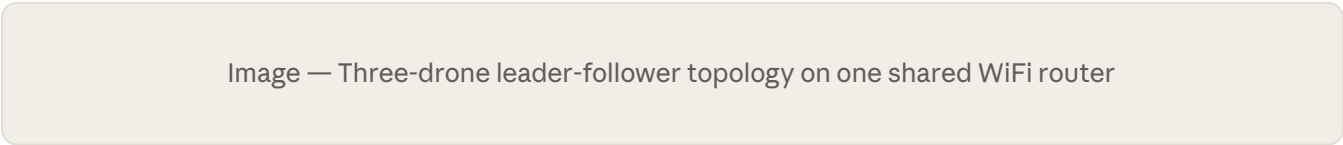
Port	Used by
RX6	FrSky Archer Plus R8 receiver

ArduPilot treats each serial port as an independent MAVLink channel with its own parameters, buffers and stream rates. Adding the Pi on SERIAL1 has no effect on the DroneBridge link on SERIAL4 — both carry telemetry simultaneously, and both can send commands. Keep DroneBridge as the only ground-control link so that when something unexpected happens you know which path commanded the aircraft.

Why a companion computer at all

Two things become possible. First, the Pi can run code on the aircraft that talks to the flight controller directly, at UART speed and without the packet loss of a WiFi link. Second, once the MAVLink stream is on a network socket rather than trapped on a wire, forwarding it to another drone becomes a configuration line instead of a hardware problem. That second property is what makes the swarm work.

System topology



Everything sits on one subnet. The two traffic classes are separated by address and port rather than by separate networks: telemetry goes from each DroneBridge to its own laptop on 14550, and swarm position goes from the leader’s Pi to the follower Pis on 14555. The router is a single point of failure for both. If it drops mid-flight you lose telemetry on all three laptops at the same moment the followers stop receiving position. The FrSky links are unaffected, so manual recovery is always available — but rehearse it rather than meeting it unexpectedly.

Wiring

Five wires total: two for power from the UBEC, three for the serial link to the flight controller.

Power, from the UBEC

UBEC	Raspberry Pi
+5V	Pin 2

UBEC	Raspberry Pi
GND	Pin 6

Pins 2 and 4 are **both 5V**. Putting the UBEC's negative on pin 4 shorts the 5V rail, and the small pin numbering makes this an easy mistake. Ground pins are 6, 9, 14, 20, 25, 30, 34 and 39.

Before first use, confirm the UBEC is genuinely 3A continuous and not 3A peak. A Pi 4 with WiFi active draws roughly 1–2A steady with sharp transients on top. Feed it from the battery or PDB pads with a short, thick run (20AWG or better) — voltage drop is the usual failure, and the Pi throttles below 4.63V. A 470–1000µF low-ESR capacitor at the Pi's input helps ride out transients.

Feeding 5V into the GPIO header bypasses the Pi's USB-C input protection and polyfuse entirely. If the UBEC is adjustable, aim for 5.0–5.15V and never above 5.25V.

Serial link to the flight controller

Matek H7A3	Raspberry Pi	Signal
TX1	Pin 10 (GPIO 15, RXD)	FC → Pi telemetry
RX1	Pin 8 (GPIO 14, TXD)	Pi → FC commands
G	Pin 9 or 14	Common reference

TX goes to RX and RX goes to TX — they cross. Both sides are 3.3V logic, so no level shifter is needed. Leave the Matek's 5V pad beside TX1/RX1 unconnected; the Pi is fed from the UBEC only.

The ground wire is mandatory, even though the UBEC ground and FC ground already meet at the PDB. That shared path carries 40+ amps of motor current, and current through resistance means the Pi's 0V and the flight controller's 0V differ by a small amount that fluctuates with throttle. UART is single-ended — the receiver decides between a 1 and a 0 by comparing against its own ground — so a drifting reference corrupts data. At 921600 baud the bit period is about 1µs and there is very little margin. The classic symptom is a link that works perfectly on the bench and throws framing errors the moment the motors spin.

Twist TX, RX and GND together as a bundle, keep the run under 15cm, and route it away from ESC leads and motor phases. Cross them at 90° if you must cross at all.

The power rule that matters most

Never have USB-C and the flight battery connected to the Pi at the same time. With 5V arriving at the GPIO header there is no input-source arbitration, so the two supplies fight and the weaker one gets backfed.

For bench work, pick one arrangement and stick to it:

- **Option A** — Pi on USB-C, flight controller on USB from your laptop, flight battery disconnected. Nothing can spin. This is the safer default.
- **Option B** — flight battery connected so the Pi runs from the UBEC, USB-C unplugged.

Props off either way. In flight it is always the UBEC, never USB.

Flight controller parameters

Do all parameter work **over USB**, not DroneBridge. Mission Planner fetches parameters one at a time over MAVLink, and over a UDP WiFi link every dropped packet is a permanently missing parameter. A partial download that looks complete will mislead you. Connect a USB cable from the Matek to your laptop, select the COM port at 115200, and the download becomes fast and lossless.

If the parameter list looks sparse, do not treat a missing parameter as meaning it does not exist. Click Refresh Params and check the count at the bottom — an H7A3 running Copter should show well over 1000.

Set these

```
SERIAL1_PROTOCOL = 2      MAVLink2 on the Pi's port
SERIAL1_BAUD      = 921    921600 (use 57 to start if debugging)
SERIAL1_OPTIONS   = 0
BRD_SER1_RTSCTS   = 0      no flow control wired
```

`BRD_SER1_RTSCTS` defaults to 2 (auto-detect). Forcing 0 removes any chance of it deciding it sees flow control that is not there.

Per-drone identity — the one that must differ

```
MAV_SYSID = 1    drone 1
MAV_SYSID = 2    drone 2
MAV_SYSID = 3    drone 3
```

Set these while all three are on the bench, before any multi-drone testing. Identical builds all default to 1, and once they are bridged over the router the routing becomes ambiguous. Worse, FOLLOW mode breaks entirely: a follower told to follow sysid 1 would see its own

position messages matching that ID.

`MAV_SYSID` requires a reboot to take effect, as does any baud change.

Stream rates

```
MAV2_POSITION = 10
MAV2_EXTRA1   = 10
MAV2_EXT_STAT = 5
MAV2_RAW_SENS = 0
MAV2_RC_CHAN  = 0
MAV2_PARAMS   = 0
```

Set the ones the Pi does not need to zero. Even at 921600 there is no reason to ship raw sensor data to a companion computer.

The 4.7 renames

ArduCopter 4.7 renamed a large batch of parameters, and any tutorial or forum post written for 4.5 or 4.6 will use the old names. If a parameter does not exist, assume a rename and search the partial string rather than the full name.

Old name	4.7 name	Note
<code>SYSID_THISMAV</code>	<code>MAV_SYSID</code>	Moved out of the Copter parameter table
<code>SYSID_MYGCS</code>	<code>MAV_GCS_SYSID</code>	Matters once the Pi issues commands
<code>SR0_*</code> ... <code>SR9_*</code>	<code>MAV1_*</code> ... <code>MAV10_*</code>	Off by one: SERIAL1's rates are <code>MAV2_*</code>
<code>ARMING_CHECK</code>	<code>ARMING_SKIPCHK</code>	Bit meanings are inverted
<code>SERIALn_OPTIONS</code> bit 12	<code>MAVn_OPTIONS</code> bit 2	Ignore stream rate override
<code>WPNAV_*</code>	<code>WP_*</code>	
<code>RTL_ALT</code>	<code>RTL_ALT_M</code>	Units changed from cm to m

The `ARMING_SKIPCHK` inversion deserves attention. A badly converted value silently skips arming checks you believe are active. Check it on every drone, using Mission Planner's checkbox view rather than computing the bitmask by hand.

Optional: stop clients overriding your rates

MAVProxy sends its own stream-rate request on connection, defaulting to 4Hz, which overrides the flight controller's configured rates for that session. If you see exactly 250ms gaps in telemetry, that is what is happening — the fix in-session is `set streamrate 10` at

the MAVProxy prompt.

To make the port ignore such overrides permanently, set bit 2 of `MAV2_OPTIONS`. Reasonable for a companion-computer port where you want deterministic rates. Verify the bit meaning in Mission Planner's own description of the parameter before writing it.

Verify by reading back

Writes can silently fail. After writing and rebooting, refresh params and confirm each value individually. Do not trust the batch.

Flashing Raspberry Pi OS

The Pi has no built-in storage. The microSD card is its entire filesystem — operating system, programs, everything. The Pi is never cabled to your laptop; its USB-C port is power only. The workflow is: card into laptop, Imager writes the OS, card into Pi, Pi joins WiFi, laptop reaches it over the network via SSH.

What you need

- The microSD card (a 32GB Class 10 card works, though see the note at the end of this section)
- A card reader, or the SD adapter that came with the card
- A USB-C charger, 3A ideally, for bench setup only
- [Raspberry Pi Imager](#) on your laptop

Everything else installs on the Pi itself over the network.

Imager settings

1. **Choose Device** → Raspberry Pi 4
2. **Choose OS** → Raspberry Pi OS (other) → **Raspberry Pi OS Lite (64-bit)**
 Lite, not the desktop version — you have 2GB of RAM and no monitor, and a desktop adds background services that can interfere with real-time serial handling. Take care not to grab the 32-bit or Legacy variants, which sit in the same submenu.
3. **Choose Storage** → your card. Check the capacity matches. It gets wiped.
4. **Next** → **Edit Settings**

The customisation dialog

This is the part that makes headless setup possible. Fill in every field.

General tab:

Field	Value
Hostname	drone1 (then drone2 , drone3)
Username	drone — keep it identical across all three
Password	your choice; there is no recovery
Wireless SSID	the network your laptop is on right now
Wireless password	—
Wireless LAN country	DE
Locale	Europe/Berlin, keyboard us

Services tab: enable SSH, password authentication.

Three fields cause nearly all first-boot failures:

- **The SSID must match your laptop's network.** SSH is your only way in. If the Pi joins network A and your laptop is on network B, the Pi boots happily and is completely unreachable. University or corporate WiFi often will not work either — enterprise authentication cannot be configured here, and client isolation blocks device-to-device traffic. A phone hotspot is a reliable fallback for both machines.
- **The country code is not cosmetic.** The WiFi radio stays disabled without a valid regulatory domain.
- **SSH must be enabled.** Without it there is no way in at all.

The keyboard layout is irrelevant headless — your laptop's layout determines what you type. `us` is the pragmatic choice if you ever attach a keyboard, since `y / z` and most shell punctuation move on a German layout.

Skip Raspberry Pi Connect. It tunnels through external servers, needs internet, and your drones will fly on an isolated router.

Write

Save → **Yes** to apply → **Yes** to confirm. Let the verification step finish rather than pulling the card early; a card that failed verification produces a Pi that boots halfway and behaves strangely, which is much harder to diagnose than an obvious failure.

A note on cards

Cheap microSD cards are the leading cause of Raspberry Pi instability. If you have the choice, a SanDisk Extreme or Samsung EVO Select with an A1 or A2 rating holds up far better under the constant small writes a root filesystem generates. Worth considering before building three of these.

First boot and system upgrade

Insert and power

The microSD slot is on the underside of the board, at the opposite end from the USB ports, next to the USB-C connector. Contacts face the board. It is a friction slot, so it just slides in.

Flight battery disconnected, then USB-C in. Red LED solid means power; green LED flickering means it is reading the card. First boot takes 60–90 seconds — it expands the filesystem, applies the customisation, and reboots itself once. If the green LED never flickers, the card is not being read; reseal it.

SSH in

From PowerShell, Terminal, or your shell:

```
ssh drone@drone1.local
```

The part before `@` is the username, the part after is the hostname. Two different things: the hostname names the machine on the network, the username names an account on it. Hostnames must differ across the three drones; the username can stay the same everywhere, which keeps scripts and file paths identical.

On first connection it asks about the host's authenticity — type `yes`. Then the password.

Nothing appears as you type it, not even asterisks. You are in when you see

```
drone@drone1:~ $ .
```

If `drone1.local` will not resolve — Windows mDNS is inconsistent — find the Pi in your router's connected-devices list and use its IP instead.

If you get "connection refused", the Pi is reachable but SSH is not running; the Imager checkbox occasionally fails to take. Fix without reflashing: put the card back in your laptop, and in the `bootfs` partition create an empty file named exactly `ssh` — no extension. On Windows you will need View → Show → File name extensions to remove the

`.txt` Explorer adds. On boot the Pi sees the file, enables SSH, and deletes it. While the card is out, `firstrun.sh` or `custom.toml` in that same partition shows what your customisation actually wrote.

If you get the host-key warning after reflashing a card, that is expected — the Pi generated a new identity. Clear the stale entry with `ssh-keygen -R drone1.local` and reconnect.

Upgrade, under tmux

```
sudo apt install -y tmux
tmux
sudo apt update && sudo apt full-upgrade -y
```

Run the upgrade inside tmux. Without it, anything you launch is a child of the SSH session — a WiFi hiccup or a sleeping laptop kills the connection, and everything running inside it dies mid-execution. An interrupted `apt full-upgrade` is the classic way to corrupt the card, because it leaves `/var/lib/dpkg/info` in an inconsistent state.

With tmux the session lives on the Pi. If SSH drops, reconnect and run `tmux attach` — the upgrade kept going.

Two keystrokes worth knowing: `Ctrl+B` then `d` detaches, `Ctrl+B` then `c` opens a second window (`Ctrl+B` then `0` or `1` switches). That second window is how you inspect things without stopping what is running.

Let the upgrade finish completely. It may sit apparently idle for stretches. Do not unplug, reboot, or close the terminal until the prompt returns.

```
sudo reboot
```

Shutting down

Always:

```
sudo poweroff
```

Then watch the green LED. Once it has been dark and still for about five seconds, it is safe to unplug. Linux caches writes in RAM and flushes them periodically; pulling power mid-flush is exactly what produces `EXT4-fs error: directory index failed checksum`.

As a habit: any command taking more than a minute over WiFi should run inside tmux.

UART configuration on the Pi

This is the step that determines whether the Pi can talk to the flight controller at all. Two file edits and three checks.

Enable the good UART

```
sudo nano /boot/firmware/config.txt
```

At the very bottom, add:

```
enable_uart=1
dtoverlay=disable-bt
```

Save with `Ctrl+O`, `Enter`, then `Ctrl+X` to exit.

`enable_uart=1` switches on the serial port on GPIO pins 8 and 10, which is off by default.

`dtoverlay=disable-bt` matters more than it looks. The Pi 4 has two UARTs:

Name	Device	Quality
PL011	<code>ttAMA0</code>	Proper hardware UART, stable clock
mini-UART	<code>ttYS0</code>	Clock tied to CPU frequency

By default the good one is reserved for Bluetooth and you get the mini-UART, whose baud rate drifts as the CPU speeds up and slows down. At 921600 it drops bytes. This overlay disconnects Bluetooth and hands you the PL011.

Then:

```
sudo systemctl disable hciuart
```

On Raspberry Pi OS Lite this will likely report that the unit does not exist. That is fine — Lite has minimal Bluetooth support, so there is no service to disable. "Unit does not exist" and "command not found" mean absence, not failure.

Free the port from the kernel console

```
sudo nano /boot/firmware/cmdline.txt
```

This file is **one single long line**. Do not press `Enter` anywhere in it — a line break here can prevent booting.

Find `console=serial0,115200` and delete it, including the trailing space. Leave `console=tty1` and everything else exactly as it is.

Without this, Linux keeps a debug console open on that serial port — sending boot messages and a login prompt into the same wires your MAVLink traffic uses.

```
sudo reboot
```

The three checks

```
ls -l /dev/serial0
```

Must end with `-> ttyAMA0`. Ignore the permissions and date; read the arrow. `-> ttyS0` means the overlay did not apply — recheck `config.txt`, confirm you saved, reboot. **Do not proceed past this check.** Working with the mini-UART means chasing phantom wiring faults for an hour.

```
cat /proc/cmdline | grep -o 'console=[^ ]*'
```

Should print only `console=tty1`. Reading `/proc/cmdline` rather than `cmdline.txt` is deliberate: it shows what the kernel actually booted with, not what you intended.

```
vcgencmd get_throttled
```

Should return `throttled=0x0`. Anything else is a bitmask — a `1` in the last position means undervoltage right now, a value like `0x50000` means it happened earlier since boot.

On USB-C from a decent charger this will almost certainly be clean and tells you nothing about your flight power chain. The reading that matters is after ten minutes on the UBEC with the battery connected, ideally with motors running. Rerun it after every flight.

MAVProxy and proving the link

Install

```
sudo apt install -y python3-pip python3-dev python3-lxml
pip3 install mavproxy future setuptools --break-system-packages
```

`--break-system-packages` is needed because Debian Bookworm and later protect the system Python from pip. It sounds alarming; it only permits pip to install alongside apt's packages.

`future` and `setuptools` are not optional. MAVProxy 1.8.74 on Python 3.13 fails with `ModuleNotFoundError: No module named 'future'` without the first, and `No module named 'pkg_resources'` without the second (`pkg_resources` ships inside `setuptools`, which Python 3.13 no longer bundles).

Put pip's install location on your path:

```
echo 'export PATH=$PATH:$HOME/.local/bin' >> ~/.bashrc
source ~/.bashrc
```

Give your user serial access, then log out and back in for it to take effect:

```
sudo usermod -aG dialout drone
```

Reading pip output

Pip is verbose and full of warnings that look alarming but are not. Look for `Successfully installed` or `Installing collected packages`, and whether the command you wanted now runs. PATH warnings, dependency conflicts in unrelated packages like `types-seaborn`, and deprecation notices are all normal noise. Real failure looks like `ERROR: Could not install packages` with the install stopping.

The same applies to MAVProxy's own startup: `Failed to load module: No module named 'adsb'` and `Failed to download /SRTM3/filelist_python` are both harmless.

Test

Power up: props off, and exactly one supply on the Pi. Simplest is Pi on USB-C, flight controller on USB from your laptop, flight battery disconnected.

```
mavproxy.py --master=/dev/serial0 --baudrate 921600
```

Within seconds you should see a heartbeat, then parameters loading, and a block like:

```
Detected vehicle 1:1 on link 0
AP: ArduCopter V4.7.0 (1511f271)
AP: MatekH7A3 00410019 3231510A 31313531
AP: IMU0: fast sampling 2.0kHz/2.0kHz
AP: Frame: QUAD/X
Received 1209 parameters (ftp)
```

Then:

```
param show MAV_SYSID
```

Getting a value back is the proof. A heartbeat alone only confirms the FC→Pi wire. The parameter reply means your request travelled Pi→FC on RX1 and the answer came back FC→Pi on TX1 — both directions confirmed. On drone 2 it should return `2`, which verifies wiring and identity in one command.

A full parameter download completing in seconds is worth noting: it is the same operation that fails over DroneBridge, and it demonstrates why the UART matters.

Useful once connected:

```
status
mode
watch ATTITUDE
watch GLOBAL_POSITION_INT
param show MAV2_*
link
```

`watch` with no argument stops watching. `Ctrl+C` exits.

If `time_boot_ms` in the ATTITUDE stream steps by exactly 250, that is MAVProxy's own 4Hz stream-rate request overriding your `MAV2_*` settings, not a parameter problem. Fix it with `set streamrate 10` at the prompt, or launch with `--streamrate=10`. Your own pymavlink code will get the configured rates, since it does not send such requests.

If it fails

First, is anything arriving at all? This single test splits the diagnosis:

```
python3 -c "import serial; s=serial.Serial('/dev/serial0',921600,timeout=3);
print(repr(s.read(200)))"
```

`b''` means nothing is arriving — physical problem. Random bytes mean data is arriving at the wrong speed — baud mismatch; retry the same command at 57600.

Symptom	Cause
Nothing at all	TX/RX swapped — the cause most of the time
Heartbeat but <code>param show</code> times out	Pi→FC broken: check the RX1 joint and that you are on physical pin 8, not pin 12
Garbage characters	Baud mismatch; confirm <code>SERIAL1_BAUD</code> and that the FC was power-cycled
<code>Permission denied</code> on <code>/dev/serial0</code>	<code>dialout</code> group not yet applied — log out and back in

Symptom	Cause
Works, then drops out	Signal integrity or power; see below
link 1 down	Nothing is transmitting — check the flight controller is actually powered

The most informative single test when the link is unreliable: set `SERIAL1_BAUD = 57` on the FC, reboot it, and retry at 57600. If 57600 is stable and 921600 is not, the wiring is correct and you have a signal-integrity problem — shorten the run, twist the wires, route away from ESC leads. `SERIAL1_BAUD = 500` is a reasonable middle ground, and 921600 is generous for what a follower loop actually needs.

mavlink-router

What it is and why

mavlink-router is a program — a compiled binary at `/usr/bin/mavlink-routerd`, kept running by systemd. Not a port, not a configuration.

It exists because a serial device has exactly one owner. When a program opens `/dev/serial0` it holds it exclusively; a second program gets “device busy”. That breaks as soon as two things need the flight controller — your swarm code, a debugging session, a ground station, a logger. Network sockets have no such restriction, so mavlink-router is the one process that holds the exclusive resource and turns it into a shareable one.

It sits in a loop: read bytes from the serial port, parse them into complete MAVLink messages, copy each one out to every other endpoint, and simultaneously watch those endpoints for incoming messages to write back to the serial port. A switchboard, not an aggregator — every message is forwarded individually and unchanged. Nothing is merged, because a position from drone 2 and one from drone 3 are two different facts that the flight controller needs to see separately.

For the swarm this is what makes forwarding a config line rather than a hardware problem.

Build

```
sudo apt install -y git meson ninja-build pkg-config gcc g++ libsystemd-dev
cmake
cd ~
git clone https://github.com/mavlink-router/mavlink-router.git
cd mavlink-router
git config --global http.postBuffer 524288000
git config --global http.version HTTP/1.1
git submodule update --init --recursive --depth 1
```

Run this inside tmux — the compile takes several minutes.

The git configuration and `--depth 1` matter on a Pi over WiFi. The MAVLink headers submodule has over 54,000 objects, and a full clone reliably fails with `RPC failed; curl 92 HTTP/2 stream was not closed cleanly`. Forcing HTTP/1.1 avoids the stream cancellation; the shallow clone cuts the download by roughly 95%. If it still stalls, an Ethernet cable makes this take seconds.

Then:

```
meson setup build . -Dsystemdsystemunitdir=/lib/systemd/system
ninja -C build
sudo ninja -C build install
```

The `-Dsystemdsystemunitdir` flag is required on Debian 13. `mavlink-router`'s build file asks `pkg-config` for a variable that `systemd.pc` used to provide and `libsystemd.pc` does not, producing `ERROR: Could not get pkg-config variable and no default provided`. Supplying the option explicitly skips that lookup. Without it you will also see `Dependency "systemd" not found` even with `libsystemd-dev` installed — the package exists under a different `pkg-config` name.

If a build fails with `Dependency "X" not found`, tried `pkgconfig`, install `libX-dev`. Debian splits libraries into runtime and development packages, and compiling needs both.

Configure

```
sudo mkdir -p /etc/mavlink-router
sudo nano /etc/mavlink-router/main.conf
```

```
[General]
TcpServerPort = 5760
ReportStats = false
MavlinkDialect = ardupilotmega

[UartEndpoint fc]
Device = /dev/serial0
Baud = 921600

[UdpEndpoint local]
Mode = Normal
Address = 127.0.0.1
Port = 14550

[UdpEndpoint local2]
Mode = Normal
Address = 127.0.0.1
Port = 14551
```

The second local endpoint matters in practice: a client binding 14550 holds it exclusively, so without 14551 you cannot run swarm code and a debugging session at the same time — which is exactly what you will want.

Start

Make sure nothing else holds the serial port first:

```
pgrep -a mavproxy
pkill mavproxy
```

Then:

```
sudo systemctl enable mavlink-router
sudo systemctl start mavlink-router
systemctl status mavlink-router
```

`enable` is what makes it start at boot — in the field there is no SSH session to launch it from. A healthy start logs:

```
Opened UART [5]fc: /dev/serial0
UART [5]fc: speed = 921600
Opened UDP Client [6]local: 127.0.0.1:14550
Opened TCP Server [8] [::]:5760
```

If it fails: `journalctl -u mavlink-router -n 30 --no-pager`. The usual cause is another process still holding the serial port.

Verify

```
mavproxy.py --master=udpin:127.0.0.1:14550
```

Note `udpin`, not `udp`. Mode = `Normal` makes mavlink-router a UDP *client* — it sends toward that address expecting something bound there. `udpin` tells MAVProxy to bind and listen, which is the matching half. With `udp`: both sides send and neither listens, and you get "Waiting for heartbeat" forever.

You should see the same heartbeat as before, now arriving through the router.

`/dev/serial0` can no longer be opened directly — that is the intended behaviour.

From your laptop, Mission Planner or MAVProxy can also attach over TCP to `drone1.local:5760`.

For your own code

```
from pymavlink import mavutil

m = mavutil.mavlink_connection('udpin:127.0.0.1:14550')
m.wait_heartbeat()
msg = m.recv_match(type='GLOBAL_POSITION_INT', blocking=True)
print(msg.lat, msg.lon, msg.relative_alt)
```

That is the leader's half of the relay. `lat` and `lon` are degrees $\times 10^7$, altitudes in millimetres.

Cloning to drones 2 and 3

Finish drone 1 completely before cloning. Every piece of software on the card gets copied for free; discovering a needed change after cloning means doing it three times instead of once.

Clone the card

```
sudo poweroff
```

Wait for the green LED to go dark and stay dark, then unplug and take the card out.

Raspberry Pi Imager cannot read a card to an image file. On Windows use **Win32DiskImager** in Read mode to produce a `.img`, then write that image to the two new cards with either tool.

On each cloned Pi, first boot

```
sudo hostnamectl set-hostname drone2
sudo nano /etc/hosts          # change the 127.0.1.1 line to drone2
sudo rm /etc/ssh/ssh_host_*
sudo dpkg-reconfigure openssh-server
sudo reboot
```

Regenerating the SSH host keys matters because the clone copies drone 1's identity. Without this all three Pis present the same key, and your laptop will produce confusing warnings when you switch between them.

Then on your laptop:

```
ssh-keygen -R drone2.local
```

Per-drone flight controller work

The SD card clone handles the Pi. Each flight controller still needs its own setup.

Firmware must match exactly across all three. Not "4.7.x" — the same build. Mixed firmware in a formation means the leader and followers can behave differently in edge cases, and you will not know whether a problem is your configuration or a version difference. Check each board's version in Mission Planner's Messages tab and flash any that differ.

When flashing, confirm the board target matches. Drone 1 reports as `MatekH7A3`; the H7A3 and H743 are different chips despite the similar names, and the wrong target gives you firmware that boots and connects while failing to store parameters.

Load drone 1's parameter file, then change what differs. Save it from drone 1 over USB: Full Parameter List → Save. The file's real value is not the twenty parameters you know about — it is the frame class and type, ESC protocol, battery monitor scaling, failsafe behaviour, geofence and flight mode assignments that were configured before you started and that you would never think to check.

Mission Planner's **Compare** button is often better than Load: it shows a side-by-side diff with checkboxes, so you see exactly what changes and can exclude the calibration values.

Expect some parameters to be rejected as read-only — `MIS_TOTAL`, `FORMAT_VERSION`, `STAT_*`, and the sensor hardware IDs like `INS_ACC*ID` and `COMPASS_DEV_ID`. Those rejections are correct; the sensor IDs in particular identify the receiving board's own hardware. Check the *count* of parameters written instead: roughly a thousand is right, a handful means the load failed.

Then set the per-drone values:

```
MAV_SYSID      = 2          (3 on drone 3)
FOLL_ENABLE    = 1
FOLL_SYSID     = 1
FOLL_OFS_TYPE  = 1
FOLL_OFS_X     = -8
FOLL_OFS_Y     = -6          (+6 on drone 3)
FOLL_OFS_Z     = -3          (-5 on drone 3)
FOLL_DIST_MAX  = 100
```

Reboot after writing.

`FOLL_OFS_TYPE = 1` puts offsets in the leader's body frame, so the formation keeps its shape through turns. Offsets are metres; negative Z is up in NED. These values are deliberately generous — GPS relative error between two receivers is metres and drifts independently, so tight spacing is how you get collisions. The altitude stagger is cheap insurance against two followers converging on the same point.

Recalibrate, last. Accelerometer, compass, radio, and ESC if applicable. Calibrations must come after the parameter load, or the file overwrites them. The compass is the one worth being fussy about — do it away from metal, laptops and phones, with the GPS module in its final mounting position. A bad compass shows up as drift or circling in exactly the position-controlled modes FOLLOW depends on.

Verify each drone individually

Before assembling the swarm, on each drone:

```
mavproxy.py --master=udpin:127.0.0.1:14550
```

```
param show MAV_SYSID
```

Returning `2` and `3` respectively confirms wiring, routing and identity in one command.

Then in the HUD: tilt the drone and check the horizon follows, rotate it and check the heading tracks. Ten seconds, and it catches a bad calibration.

Per-drone configuration matrix

All parameter work over USB, never DroneBridge.

What must be identical on all three

These come from drone 1's saved parameter file and must not differ:

```
Firmware version           same build, not just "4.7.x"
FRAME_CLASS / FRAME_TYPE
MOT_PWM_TYPE and ESC settings
BATT_MONITOR and scaling
SERIAL1_PROTOCOL = 2
SERIAL1_BAUD      = 921
BRD_SER1_RTSCTS  = 0
SERIAL4_PROTOCOL = 2      DroneBridge
SERIAL4_BAUD      match drone 1
MAV2_POSITION    = 10
MAV2_EXTRA1      = 10
MAV2_EXT_STAT    = 5
MAV2_RAW_SENS    = 0
MAV2_RC_CHAN     = 0
MAV2_PARAMS      = 0
Failsafe settings, geofence, flight mode switch assignments
```

A formation where the aircraft respond differently to the same conditions is much harder to reason about. Load the file rather than hand-typing, because the parameters that matter most are the ones you would never think to check.

What differs

Parameter	Drone 1	Drone 2	Drone 3
MAV_SYSID	1	2	3
FOLL_ENABLE	0	1	1
FOLL_SYSID	—	1	1
FOLL_OFS_TYPE	—	1	1
FOLL_OFS_X	—	-8	-8
FOLL_OFS_Y	—	-6	+6
FOLL_OFS_Z	—	-3	-5
FOLL_DIST_MAX	—	100	100

Parameter	Drone 1	Drone 2	Drone 3
FOLL_YAW_BEHAVE	—	match drone 3	match drone 2

MAV_SYSID requires a reboot. Everything else takes effect immediately, though rebooting after a batch of writes is good practice.

Reading the offsets

Offsets are in metres, in the leader's body frame because FOLL_OFS_TYPE = 1. Negative X is behind, negative Y is left, **negative Z is up** (the frame is North-East-Down).

So drone 2 sits 8m behind, 6m left and 3m above the leader; drone 3 sits 8m behind, 6m right and 5m above. A trailing V with the two followers at different altitudes.

Body-frame offsets rotate with the leader's heading, so the formation keeps its shape through turns. FOLL_OFS_TYPE = 0 would fix the offsets to compass north instead, which looks strange when the leader turns.

These spacings are deliberately generous. GPS relative error between two independent receivers is metres and drifts independently, so the position a follower is chasing is only accurate to a few metres regardless of how fast you send it. Tight formations need RTK on all three, which is a different budget. The altitude stagger is cheap insurance: if horizontal control degrades badly, the two followers are still at different heights.

For a first flight consider widening to 15m and tightening once you have watched the behaviour.

Calibrations — per airframe, never copied

Accelerometer, compass, radio and ESC calibration belong to each aircraft's own sensors and receiver. Do them **after** loading the parameter file, or the file overwrites them.

The compass is the one to be fussy about: away from metal, laptops and phones, with the GPS module in its final mounting position. A bad compass appears as drift or circling in exactly the position-controlled modes FOLLOW depends on.

DroneBridge settings per drone

Configured through each ESP32's own web interface. Connect to its access point once, open it in a browser (usually 192.168.4.1), change the settings, save. From then on it joins your router on boot and you never touch its AP again.

Station mode, one laptop per drone

Setting	Drone 1	Drone 2	Drone 3
Mode	station / client	station / client	station / client
SSID	swarm router	swarm router	swarm router
Its own IP (DHCP reservation)	192.168.1.111	192.168.1.112	192.168.1.113
Target IP	192.168.1.100	192.168.1.104	192.168.1.105
Target port	14550	14550	14550
UART baud	match <code>SERIAL4_BAUD</code>	same	same
WiFi channel	1	6	11

Station mode means the ESP32 joins your existing network rather than creating its own. That is what lets each laptop hold a single WiFi connection and still reach both its drone's telemetry and any Pi over SSH.

Each DroneBridge sends its MAVLink to a different laptop's IP, so each operator sees only their own aircraft. The separation is by addressing, not by separate networks.

The channel spread matters: the ESP32-C6 is 2.4GHz only, so three DroneBridges plus three FrSky links share that band. Channels 1, 6 and 11 are the non-overlapping set.

Alternative: independent access points

If you would rather each telemetry link be fully independent of the router, leave the DroneBridges in AP mode and give each a distinct SSID — `DB_drone1` , `DB_drone2` , `DB_drone3` . Each laptop joins one AP.

The cost: a laptop on a DroneBridge AP is on a different subnet from the Pis, so it cannot SSH into them. You would need Ethernet to the router as a second interface. Two WiFi adapters on one laptop is not a reliable substitute — the drivers fight, and the two radios interfere at close range.

Suggested approach: station mode during bring-up and debugging, when you need SSH access constantly. Switch to independent APs later if you want the failure independence for a demonstration. It is a config change in the web interface, not a rebuild.

Router settings

Static DHCP reservations by MAC for all nine devices — three Pis, three DroneBridges, three laptops. Get a Pi's MAC with `ip link show wlan0`; the DroneBridge shows its own in the web interface.

Put the Pis on **5GHz** if the router is dual-band. That removes three emitters from the 2.4GHz band and separates swarm traffic from telemetry entirely.

Test the actual range where you will fly. A consumer router covers far less ground than expected, and a Pi losing the network mid-flight must be a survivable event in your design.

A useful extra

If the firmware allows more than one telemetry destination, add a fourth laptop's IP to all three DroneBridges — or simply connect one Mission Planner to all three Pis over TCP 5760. Because Mission Planner separates vehicles by system ID, that gives you a single map with the whole formation on it, which is the view that actually tells you whether FOLLOW is working.

Raspberry Pi settings per drone

Identity

	Drone 1	Drone 2	Drone 3
Hostname	drone1	drone2	drone3
IP (DHCP reservation)	192.168.1.101	192.168.1.102	192.168.1.103
Username	drone	drone	drone

The username stays identical so scripts and file paths are the same everywhere. Hostnames must differ.

On each clone:

```
sudo hostnamectl set-hostname drone2
sudo nano /etc/hosts          # change the 127.0.1.1 line
sudo rm /etc/ssh/ssh_host_*
sudo dpkg-reconfigure openssh-server
sudo reboot
```

mavlink-router config — the leader

`/etc/mavlink-router/main.conf` on drone 1:

```
[General]
TcpServerPort = 5760
ReportStats = false
MavlinkDialect = ardupilotmega

[UartEndpoint fc]
Device = /dev/serial0
Baud = 921600

[UdpEndpoint local]
Mode = Normal
Address = 127.0.0.1
Port = 14550

[UdpEndpoint local2]
Mode = Normal
Address = 127.0.0.1
Port = 14551

[UdpEndpoint follower2]
Mode = Normal
Address = 192.168.1.102
Port = 14555

[UdpEndpoint follower3]
Mode = Normal
Address = 192.168.1.103
Port = 14555
```

mavlink-router config — the followers

Drones 2 and 3 get the same file **without** the two follower endpoints, and **with** a listener instead:

```
[UdpEndpoint fromleader]
Mode = Server
Address = 0.0.0.0
Port = 14555
```

`Mode = Normal` makes an endpoint a UDP client that sends to a fixed address; `Mode = Server` binds a local port and waits. The leader pushes, the followers listen.

`0.0.0.0` means accept on any interface — you do not name the Pi's own address.

Using 14555 rather than 14550 for the inter-drone flow keeps it distinguishable from local traffic in logs and packet captures. Worth the small effort when something misbehaves.

After editing:

```
sudo systemctl restart mavlink-router
systemctl status mavlink-router
```

The clone gotcha

Cloning drone 1's SD card copies its `main.conf` — **including the two follower endpoints**. On drone 2 that means it forwards its own stream to itself and to drone 3.

Replace `/etc/mavlink-router/main.conf` on both followers before flying. The symptom if you forget is confusing: followers receiving position messages from each other as well as from the leader.

The same applies to anything else drone-specific you add later — scripts, service files, cron entries. Check what the clone brought with it.

Role is defined in two places

The flight controller parameters decide the role: `FOLL_ENABLE` and `FOLL_SYSID` are what make an aircraft a follower.

The mavlink-router config decides where messages flow. It does not declare a role in words — it is asymmetric because leader-follower is asymmetric.

Swapping roles therefore means editing both. This is a fixed-role design, which is the right choice for a demonstration. Runtime-assignable roles would need a Python layer managing its own config, and that is a natural next step if the swarm work grows beyond this.

The two operating modes

The good news: these are almost the same configuration. Solo flight is not a separate setup — it is the swarm setup with FOLLOW unused.

Mode A — solo flight, one drone at a time

This is how every drone should be tested before any of them fly together.

The Pi is not needed at all. The flight controller flies the aircraft; the Pi only relays MAVLink. You can leave it unpowered, or powered and idle, and the drone behaves identically. Nothing in the airframe depends on it.

What you need:

- Flight controller powered, props on, RC transmitter bound
- DroneBridge for telemetry (or nothing, if you are flying visually)
- `FOLL_ENABLE = 0`, or simply never select FOLLOW on the mode switch

Fly in **Loiter**, not Stabilize. Loiter holds position, which means you can leave the aircraft hovering while you attend to something else — and it exercises exactly the position controller and EKF that FOLLOW depends on. A drone that drifts or hunts in Loiter will follow badly; that is the most informative single test you can do.

What to check on each drone solo:

- Clean position hold in Loiter
- Mode switching works and recovers the aircraft instantly
- With the Pi powered and MAVProxy watching, no UART dropouts during flight
- `vcgencmd get_throttled` returns `0x0` afterwards
- Vibration levels and EKF innovations look sane in the `.bin` log

Mode B — leader-follower

Everything from Mode A, plus:

- All three Pis powered and on the router
- The leader's `main.conf` forwarding to both followers
- Each follower's `main.conf` listening on 14555
- `FOLL_ENABLE = 1` on the followers, with their offsets set
- FOLLOW assigned to a position on each follower's mode switch

Switching between them

Nothing needs rewriting. To fly a drone solo that is configured as a follower, just do not select FOLLOW — Loiter and Stabilize work exactly as before. Optionally set `FOLL_ENABLE = 0` so an accidental mode selection does nothing.

That is the whole difference. The FOLLOW parameters sit dormant until the mode is selected.

The launch sequence for Mode B

One person per drone, each with a transmitter, thumb on the mode switch. This is not a single-operator activity.

1. **Leader up first**, stable hover in Loiter at working altitude. Leave it.
2. **Follower 1 up**, in Loiter, at roughly its configured `FOLL_OFS_Z` altitude so that switching modes does not produce a large vertical correction.
3. **Verify it is receiving the leader's position before switching**. On that follower's Pi:

```
mavproxy.py --master=udpin:127.0.0.1:14551
watch GLOBAL_POSITION_INT
```

You should see messages from system 1 as well as its own. No leader messages means FOLLOW will do nothing useful. This is the step people skip.

4. **Switch to FOLLOW**. Watch it move to station and settle.
5. **Only then launch follower 2** and repeat.

One at a time is slower, but when something goes wrong you know which drone caused it.

Failures to rehearse deliberately

Before flying all three, fly the leader and one follower and break things on purpose:

- **Stop the leader's router mid-flight** (`sudo systemctl stop mavlink-router`) and watch what the follower does with stale position
- **Power off the WiFi router** — this kills telemetry on all three laptops and the swarm link simultaneously. Confirm you can recover every aircraft on the mode switch
- **Walk the leader out of WiFi range**
- **Switch a follower back to Loiter** and confirm it responds immediately

That last one is the check that matters most. If the mode switch does not reliably recover the aircraft, stop and fix that before adding a third drone.

All of these are worth doing in SITL first, where they cost nothing.

Troubleshooting

Every failure encountered on the first build, with what actually fixed it.

Symptom	Cause and fix
connection refused on SSH	SSH not running — the Imager checkbox did not take. Create an empty file named exactly <code>ssh</code> in the <code>bootfs</code> partition, no extension
REMOTE HOST IDENTIFICATION HAS CHANGED	Expected after reflashing. <code>ssh-keygen -R drone1.local</code>
hciuart.service does not exist	Normal on Lite — no Bluetooth service to disable. Ignore
EXT4-fs error: directory index failed checksum	Filesystem corruption from an interrupted write. Not repairable in place; reflash. Prevent with <code>tmux</code> and <code>sudo poweroff</code>
dpkg: bad message / sub-process returned error code 2	Same corruption. Reflash
ModuleNotFoundError: No module named 'future'	<code>pip3 install future --break-system-packages</code>
ModuleNotFoundError: No module named 'pkg_resources'	<code>pip3 install setuptools --break-system-packages</code>
Failed to load module: No module named 'adsb'	Optional MAVProxy module. Ignore
Failed to download /SRTM3/filelist_python	Terrain data needs internet. Ignore
link 1 down	Nothing is transmitting. Check the flight controller is actually powered — the Pi being on does not power the FC
Waiting for heartbeat with udp:	Use <code>udpin:</code> — both sides were sending, neither listening
RPC failed; curl 92 HTTP/2 stream was not closed cleanly	Large git clone over flaky WiFi. Force HTTP/1.1 and use <code>--depth 1</code>
Dependency "systemd" not found, tried pkgconfig	Installed as <code>libsystemd</code> , not <code>systemd</code> . Verify with <code>pkg-config --modversion libsystemd</code>
Could not get pkg-config variable and no default provided	Debian 13 packaging. Pass <code>-Dsystemdsystemunitdir=/lib/systemd/system</code> to meson

Symptom	Cause and fix
Neither source directory 'build' nor build directory . contain meson.build	Wrong directory. <code>cd ~/mavlink-router</code> and confirm with <code>ls meson.build</code>
ATTITUDE arriving at exactly 250ms intervals	MAVProxy's own 4Hz stream request overriding the FC. <code>set streamrate 10</code>
Mission Planner parameter list incomplete	UDP packet loss over DroneBridge. Use USB for all parameter work
Parameter shows as missing	Likely a 4.7 rename, or an incomplete download. Search the partial string; never treat absence as meaningful
MIS_TOTAL is read only and similar during a parameter load	Correct behaviour for firmware-maintained values. Check the written count instead

Two diagnostic principles

"Unit does not exist" and "command not found" mean absence, not failure. The errors that matter are ones where something you need refuses to work.

Isolate before fixing. When a link fails, one test splits the diagnosis cleanly:

```
python3 -c "import serial; s=serial.Serial('/dev/serial0',921600,timeout=3);
print(repr(s.read(200)))"
```

Empty means nothing is arriving — physical. Random bytes mean wrong speed — baud. Similarly, when parameter writes fail, try writing one parameter alone: if a single write fails, the board cannot store parameters at all and retrying the batch will not help.

Most build failures on Linux are either a missing `-dev` package or being in the wrong directory. `pwd` and `ls` answer the second in two seconds.

Outstanding before flight

The procedure above produces a working Pi. These four items are not yet done and each matters.

The noise test

Everything so far was verified at rest. The failure this catches only appears with motors running.

Props off, flight battery connected, USB-C unplugged from the Pi. Then:

```
mavproxy.py --master=udpin:127.0.0.1:14550 --streamrate=10
```

```
watch ATTITUDE
```

Run the motors at low throttle and watch for stuttering, gaps in `time_boot_ms`, or `link 1 down`. Afterwards:

```
vcgencmd get_throttled
```

Two failure modes: UART wiring picking up ESC noise (reroute and twist the wires), and the UBEC sagging under motor load (check the supply rating, add a capacitor at the Pi's input). Both are trivial on a bench and nasty at altitude.

This needs a GPS fix to arm, so it happens outdoors. Do not reach for `ARMING_SKIPCHK` to work around that — its bits are inverted from the old `ARMING_CHECK` and it is easy to disable more than intended.

overlayroot

Disconnecting a flight battery cuts the Pi's power instantly, with no chance to flush writes. That is exactly what corrupted the first card, and it will happen after every flight.

`overlayroot` makes the root filesystem read-only, with writes going to RAM and discarded on reboot. The card is never written to in flight, so it cannot corrupt. This is standard practice for drone companion computers.

Set it up after the swarm software works — a read-only filesystem makes development awkward — but before flying regularly, or you will be reflashing cards.

Static IPs

In the router's admin page, add DHCP reservations by MAC address. Get each Pi's MAC with `ip link show wlan0`.

```
drone1 → 192.168.1.101
drone2 → 192.168.1.102
drone3 → 192.168.1.103
laptop → 192.168.1.100
```

Use the 5GHz band if the router and Pis support it — 2.4GHz collides with the FrSky RC link. Test the actual range where you will fly; a consumer router on a tripod covers much less ground than you would expect, and a Pi losing the network mid-flight needs to be a survivable event in your design.

WiFi networks can be added later without reflashing: `sudo nmcli device wifi connect "SSID" password "..."` saves it permanently, and NetworkManager joins whichever is in range. Use `connection.autoconnect-priority` if you want one preferred.

SITL testing

Before flying three real aircraft, run the same parameters in simulation. ArduPilot SITL with Gazebo lets you fly three simulated copters on a laptop and watch the formation form — confirming the offsets produce the geometry you expect, seeing how followers respond when the leader turns, and checking yaw behaviour.

Most importantly, test the failure case deliberately: kill the leader's router mid-flight and observe what a follower does when the leader's position goes stale. UDP has no delivery guarantee, and that scenario is the one most likely to hurt you in a field. Your laptop is a much cheaper place to find the answer.

On Windows this needs WSL2, which is a project in itself, but it is the difference between debugging formation geometry in simulation and debugging it in the air.

Then the swarm layer

Once all three drones reach this point, the remaining work is small. The leader's `main.conf` gains a UDP endpoint per follower:

```
[UdpEndpoint follower2]
Mode = Normal
Address = 192.168.1.102
Port = 14555
```

And each follower gains a listener:

```
[UdpEndpoint fromleader]
Mode = Server
Address = 0.0.0.0
Port = 14555
```

Use a separate port from the local 14550 so the two flows stay distinguishable in logs and packet captures.

With that, the leader's `GLOBAL_POSITION_INT` reaches each follower's flight controller, which matches it against `FOLL_SYSID` and follows. **Possibly no Python at all** for the baseline.

A filter script that forwards only `GLOBAL_POSITION_INT` at a controlled rate is the obvious optimisation — forwarding the whole stream wastes WiFi bandwidth on attitude, battery and status messages the follower does not need, and limits what can surprise you. Worth doing, but after the baseline works.

The real constraints on formation quality are latency (20–100ms across the WiFi hop, with jitter mattering more than the average) and GPS relative error between independent receivers, which is metres and drifts. Fly the leader gently, and keep the spacing generous.